Android Developer Task: FCM Topic Broadcast & Notification System

> ⚠️ **STRICT DIRECTIVE FOR DEVELOPER:**
> DO NOT modify, remove, or refactor any existing app features, UI components, Canvas / Poster Editor, Templates, In-App Purchases, Ads (AdMob / Facebook Audience Network), or OneSignal configuration.
> **ONLY add or update the Firebase Cloud Messaging (FCM) Push Notification components** as detailed below.

📌 **OBJECTIVE:** Our backend publishes automated daily push notifications via Firebase Cloud Messaging (FCM HTTP v1 API) using topic broadcasting. Every installed app must:

1. Automatically subscribe to the global FCM topic `"all_users"` upon launch so all devices receive automated daily slot notifications.
2. Register its FCM device token with our backend server API.
3. Display incoming notifications with rich preview images (`BigPictureStyle`) in both background and foreground states.
4. Persist notifications into the in-app notification center database (`MailER_NotificationModel`).
5. Send delivery telemetry (`DISPLAYED`) and click telemetry (`OPENED`) to the backend.

2. Step-by-Step Implementation

Step 1 Update `app/src/main/AndroidManifest.xml`

1. Ensure notification permission is declared:

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

2. Inside `<application>` , set default notification channel ID and register the FCM service:

```
<!-- Default Notification Channel for FCM -->
<meta-data
    android:name="com.google.firebase.messaging.default_notification_channel_id"
    android:value="postermaker_general" />

<!-- FCM Messaging Service -->
<service
    android:name=".utils.PostermakerFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

Step 2 Create `PostermakerFirebaseMessagingService.java`

Create file: `app/src/main/java/com/festival/flyer/postermaker/utils/PostermakerFirebaseMessagingService.java`

```
package com.festival.flyer.postermaker.utils;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Build;
import android.provider.Settings;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;

import com.festival.flyer.postermaker.R;
import com.festival.flyer.postermaker.activities.MailER_SplashScreen;
import com.festival.flyer.postermaker.models.MailER_NotificationModel;
import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;

import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Map;

public class PostermakerFirebaseMessagingService extends FirebaseMessagingService {

    private static final String TAG = "PostermakerFCM";
    private static final String CHANNEL_ID = "postermaker_general";
    private static final String CHANNEL_NAME = "Poster Maker Updates";

    @Override
    public void onNewToken(@NonNull String token) {
        super.onNewToken(token);
        Log.d(TAG, "New FCM Token received: " + token);
        registerTokenWithBackend(token);
    }

    @Override
    public void onMessageReceived(@NonNull RemoteMessage remoteMessage) {
        super.onMessageReceived(remoteMessage);
        Log.d(TAG, "FCM message received from: " + remoteMessage.getFrom());

        Map<String, String> data = remoteMessage.getData();
        RemoteMessage.Notification notification = remoteMessage.getNotification();

        // 1. Extract content safely from notification payload or data payload
        String title = "Poster Maker";
        String body = "";
        String imageUrl = null;
        String campaignId = null;
        String targetScreen = "home";
        String catId = "10";
        String notifId = String.valueOf(System.currentTimeMillis());
```

```
        if (notification != null) {
            if (notification.getTitle() != null && !notification.getTitle().isEmpty()) {
                title = notification.getTitle();
            }
            if (notification.getBody() != null && !notification.getBody().isEmpty()) {
                body = notification.getBody();
            }
            if (notification.getImageUrl() != null) {
                imageUrl = notification.getImageUrl().toString();
            }
        }

        if (data != null && !data.isEmpty()) {
            if (data.containsKey("title") && (notification == null || notification.getTitle() == null))
{
                title = data.get("title");
            }
            if (data.containsKey("body") && (notification == null || notification.getBody() == null)) {
                body = data.get("body");
            }
            if (data.containsKey("message") && (body == null || body.isEmpty())) {
                body = data.get("message");
            }
            if (data.containsKey("image_url") && (imageUrl == null || imageUrl.isEmpty())) {
                imageUrl = data.get("image_url");
            }
            if (data.containsKey("big_picture") && (imageUrl == null || imageUrl.isEmpty())) {
                imageUrl = data.get("big_picture");
            }
            if (data.containsKey("image") && (imageUrl == null || imageUrl.isEmpty())) {
                imageUrl = data.get("image");
            }
            if (data.containsKey("campaign_id")) {
                campaignId = data.get("campaign_id");
            }
            if (data.containsKey("notification_id")) {
                notifId = data.get("notification_id");
            }
            if (data.containsKey("target_screen")) {
                targetScreen = data.get("target_screen");
            }
            if (data.containsKey("cat_id")) {
                catId = data.get("cat_id");
            }
        }

        // 2. Persist notification to internal app history
        MailER_NotificationModel model = new MailER_NotificationModel(
                notifId,
                title,
                body,
                imageUrl,
                System.currentTimeMillis()
        );
        MailER_NotificationHelper.saveNotification(getApplicationContext(), model);

        // 3. Display System Notification with rich BigPictureStyle
```

```
        showNotification(title, body, imageUrl, campaignId, targetScreen, catId, notifId);

        // 4. Send Confirmed Delivery Telemetry (DISPLAYED) to server
        if (campaignId != null && !campaignId.trim().isEmpty()) {
            MailER_NotificationHelper.trackNotificationEvent(getApplicationContext(), campaignId,
"DISPLAYED");
        }
    }

    private void showNotification(String title, String body, String imageUrl, String campaignId,
                                  String targetScreen, String catId, String notifId) {
        createNotificationChannel();

        Intent intent = new Intent(this, MailER_SplashScreen.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_SINGLE_TOP);
        intent.putExtra("is_from_notification", true);
        intent.putExtra("campaign_id", campaignId != null ? campaignId : "");
        intent.putExtra("target_screen", targetScreen != null ? targetScreen : "");
        intent.putExtra("cat_id", catId != null ? catId : "");
        intent.putExtra("title", title);
        intent.putExtra("body", body);
        intent.putExtra("image_url", imageUrl != null ? imageUrl : "");

        int flags = PendingIntent.FLAG_UPDATE_CURRENT;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            flags |= PendingIntent.FLAG_IMMUTABLE;
        }

        int reqCode = (int) (System.currentTimeMillis() % 100000);
        PendingIntent pendingIntent = PendingIntent.getActivity(this, reqCode, intent, flags);

        NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID)
                .setSmallIcon(R.drawable.spawner_ic_notification)
                .setContentTitle(title)
                .setContentText(body)
                .setAutoCancel(true)
                .setPriority(NotificationCompat.PRIORITY_HIGH)
                .setContentIntent(pendingIntent);

        // Download rich image for BigPictureStyle preview
        Bitmap bitmap = null;
        if (imageUrl != null && !imageUrl.trim().isEmpty()) {
            bitmap = downloadBitmap(imageUrl);
        }

        if (bitmap != null) {
            builder.setLargeIcon(bitmap);
            builder.setStyle(new NotificationCompat.BigPictureStyle()
                    .bigPicture(bitmap)
                    .setSummaryText(body));
        } else {
            builder.setStyle(new NotificationCompat.BigTextStyle().bigText(body));
        }

        NotificationManagerCompat manager = NotificationManagerCompat.from(this);
        try {
            manager.notify(reqCode, builder.build());
        } catch (SecurityException se) {
```

```
            Log.e(TAG, "Notification permission not granted: " + se.getMessage());
        } catch (Exception e) {
            Log.e(TAG, "Error posting notification: " + e.getMessage());
        }
    }

    private Bitmap downloadBitmap(String urlStr) {
        HttpURLConnection connection = null;
        try {
            URL url = new URL(urlStr);
            connection = (HttpURLConnection) url.openConnection();
            connection.setDoInput(true);
            connection.setConnectTimeout(6000);
            connection.setReadTimeout(6000);
            connection.connect();
            InputStream input = connection.getInputStream();
            return BitmapFactory.decodeStream(input);
        } catch (Exception e) {
            Log.w(TAG, "Failed to download notification image: " + e.getMessage());
            return null;
        } finally {
            if (connection != null) {
                try {
                    connection.disconnect();
                } catch (Exception ignored) {}
            }
        }
    }

    private void createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel channel = new NotificationChannel(
                    CHANNEL_ID,
                    CHANNEL_NAME,
                    NotificationManager.IMPORTANCE_HIGH
            );
            channel.setDescription("Scheduled and promotional updates from Poster Maker");
            channel.enableVibration(true);
            NotificationManager manager = getSystemService(NotificationManager.class);
            if (manager != null) {
                manager.createNotificationChannel(channel);
            }
        }
    }

    private void registerTokenWithBackend(String token) {
        new Thread(() -> {
            HttpURLConnection conn = null;
            try {
                String deviceId = Settings.Secure.getString(getContentResolver(),
Settings.Secure.ANDROID_ID);
                URL url = new URL("https://cygnux.in/postermaker/API/V1/DeviceToken/register");
                conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("POST");
                conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
                conn.setConnectTimeout(5000);
                conn.setReadTimeout(5000);
                conn.setDoOutput(true);
```

```
                String postData = "device_id=" + java.net.URLEncoder.encode(deviceId != null ? deviceId
: "unknown", "UTF-8")
                        + "&fcm_token=" + java.net.URLEncoder.encode(token, "UTF-8")
                        + "&app_version=" +
java.net.URLEncoder.encode(com.festival.flyer.postermaker.BuildConfig.VERSION_NAME, "UTF-8")
                        + "&language=en"
                        + "&is_test_device=" + (com.festival.flyer.postermaker.BuildConfig.DEBUG ? "1" :
"0");

                try (java.io.OutputStream os = conn.getOutputStream()) {
                    byte[] input = postData.getBytes(java.nio.charset.StandardCharsets.UTF_8);
                    os.write(input, 0, input.length);
                }

                int code = conn.getResponseCode();
                Log.d(TAG, "Token registered with backend HTTP code: " + code);
            } catch (Exception e) {
                Log.e(TAG, "Failed to register token with backend: " + e.getMessage());
            } finally {
                if (conn != null) {
                    try {
                        conn.disconnect();
                    } catch (Exception ignored) {}
                }
            }
        }).start();
    }
}
```

Step 3 Update `MyApplication.java`

In `app/src/main/java/com/festival/flyer/postermaker/MyApplication.java` inside `onCreate()`:

```
 // Create Global Push Notification Channel (Android 8.0+)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    android.app.NotificationChannel channel = new android.app.NotificationChannel(
            "postermaker_general",
            "Poster Maker Updates",
            android.app.NotificationManager.IMPORTANCE_HIGH
    );
    channel.setDescription("Poster Maker Push Notifications");
    channel.enableVibration(true);
    android.app.NotificationManager manager = getSystemService(android.app.NotificationManager.class);
    if (manager != null) {
        manager.createNotificationChannel(channel);
    }
}
```

Step 4 Update `MailER_NotificationHelper.java`

In `app/src/main/java/com/festival/flyer/postermaker/utils/MailER_NotificationHelper.java`, add the telemetry method:

```
/**
 * Send telemetry event (DISPLAYED or OPENED) to backend API asynchronously.
 */
public static void trackNotificationEvent(Context context, String campaignId, String eventType) {
    if (context == null || campaignId == null || campaignId.trim().isEmpty() || eventType == null) {
        return;
    }

    new Thread(() -> {
        java.net.HttpURLConnection conn = null;
        try {
            String deviceId = android.provider.Settings.Secure.getString(
                    context.getContentResolver(),
                    android.provider.Settings.Secure.ANDROID_ID
            );
            if (deviceId == null || deviceId.isEmpty()) {
                deviceId = "android_" + System.currentTimeMillis();
            }

            java.net.URL url = new
java.net.URL("https://cygnux.in/postermaker/API/V1/DeviceToken/track_notification");
            conn = (java.net.HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
            conn.setConnectTimeout(5000);
            conn.setReadTimeout(5000);
            conn.setDoOutput(true);

            String postData = "campaign_id=" + java.net.URLEncoder.encode(campaignId.trim(), "UTF-8")
                    + "&device_id=" + java.net.URLEncoder.encode(deviceId.trim(), "UTF-8")
                    + "&event_type=" + java.net.URLEncoder.encode(eventType.trim().toUpperCase(), "UTF-
8");

            try (java.io.OutputStream os = conn.getOutputStream()) {
                byte[] input = postData.getBytes(java.nio.charset.StandardCharsets.UTF_8);
                os.write(input, 0, input.length);
            }

            int code = conn.getResponseCode();
            android.util.Log.d("FCM_TELEMETRY", "Event: " + eventType + " for campaign " + campaignId +
" tracked with HTTP code: " + code);
        } catch (Exception e) {
            android.util.Log.e("FCM_TELEMETRY", "Failed to track " + eventType + " event: " +
e.getMessage());
        } finally {
            if (conn != null) {
                try {
                    conn.disconnect();
                } catch (Exception ignored) {}
            }
        }
    }).start();
}
```

Step 5 Update `MailER_PosterMainActivity.java`

In `app/src/main/java/com/festival/flyer/postermaker/activities/MailER_PosterMainActivity.java`:

1. In `onCreate()`, add topic subscription, token registration, and notification intent handling:

```
 // Subscribe to General Push Topic (Reaches all users via 1 FCM HTTP v1 broadcast)
com.google.firebase.messaging.FirebaseMessaging.getInstance().subscribeToTopic("all_users")
        .addOnCompleteListener(topicTask -> {
            if (topicTask.isSuccessful()) {
                android.util.Log.d("FCM_TOPIC", "Subscribed to 'all_users' topic successfully");
            }
        });

// Direct FCM Token Logging & Server Registration
com.google.firebase.messaging.FirebaseMessaging.getInstance().getToken().addOnCompleteListener(task -> {
    if (task.isSuccessful()) {
        String fcmToken = task.getResult();
        android.util.Log.d("FCM_TOKEN", "Direct FCM Token: " + fcmToken);
        registerFCMTokenWithBackend(fcmToken);
    }
});

// Handle push notification launch click
handleNotificationIntent(getIntent());
```

2. Add these helper methods to `MailER_PosterMainActivity`:

```
 private void registerFCMTokenWithBackend(String token) {
    if (token == null || token.isEmpty()) return;
    try {
        String deviceId = android.provider.Settings.Secure.getString(getContentResolver(),
android.provider.Settings.Secure.ANDROID_ID);
        String url = "https://cygnux.in/postermaker/API/V1/DeviceToken/register";

        com.android.volley.toolbox.StringRequest request = new com.android.volley.toolbox.StringRequest(
                com.android.volley.Request.Method.POST, url,
                response -> android.util.Log.d("FCM_BACKEND", "FCM token registered with backend: " +
response),
                error -> android.util.Log.e("FCM_BACKEND", "Failed to register FCM token with backend",
error)
        ) {
            @Override
            protected java.util.Map<String, String> getParams() {
                java.util.Map<String, String> params = new java.util.HashMap<>();
                params.put("device_id", deviceId != null ? deviceId : "unknown_android_" +
System.currentTimeMillis());
                params.put("fcm_token", token);
                params.put("app_version", com.festival.flyer.postermaker.BuildConfig.VERSION_NAME);
                params.put("language", "en");
                params.put("is_test_device", com.festival.flyer.postermaker.BuildConfig.DEBUG ? "1" :
"0");
                return params;
            }
        };
        com.android.volley.toolbox.Volley.newRequestQueue(this).add(request);
    } catch (Exception e) {
        android.util.Log.e("FCM_BACKEND", "Error registering FCM token", e);
    }
}

@Override
protected void onNewIntent(Intent intent) {
    super.onNewIntent(intent);
    setIntent(intent);
    handleNotificationIntent(intent);
}

private void handleNotificationIntent(Intent intent) {
    if (intent == null || !intent.getBooleanExtra("is_from_notification", false)) {
        return;
    }

    String campaignId = intent.getStringExtra("campaign_id");
    if (campaignId != null && !campaignId.trim().isEmpty()) {
        com.festival.flyer.postermaker.utils.MailER_NotificationHelper.trackNotificationEvent(this,
campaignId, "OPENED");
    }

    String targetScreen = intent.getStringExtra("target_screen");
    if (targetScreen != null) {
        targetScreen = targetScreen.trim().toLowerCase();
        if (targetScreen.equals("notifications") || targetScreen.equals("notification_center")) {
            nextActivity(MailER_NotificationActivity.class);
        } else if (targetScreen.equals("create")) {
```

```
            nextActivity(MailER_CreateActivity.class);
        } else if (targetScreen.equals("custom_category") || targetScreen.equals("category_detail")) {
            nextActivity(MailER_CustomCategoryActivity.class);
        } else if (targetScreen.equals("business_profile")) {

nextActivity(com.festival.flyer.postermaker.makeityours.auto.MailER_BusinessProfileActivity.class);
        }
    }
}
```

Step 6 Update `MailER_SplashScreen.java`

In `app/src/main/java/com/festival/flyer/postermaker/activities/MailER_SplashScreen.java`:

1. In `onCreate()`, track telemetry if launched from notification:

```
Intent launchIntent = getIntent();
if (launchIntent != null && launchIntent.getBooleanExtra("is_from_notification", false)) {
    String campaignId = launchIntent.getStringExtra("campaign_id");
    if (campaignId != null && !campaignId.trim().isEmpty()) {
        MailER_NotificationHelper.trackNotificationEvent(this, campaignId, "OPENED");
    }
}
```

2. Forward intent extras when transitioning from Splash to main activity:

```
private Intent createForwardIntent(Class<?> targetActivity) {
    Intent intent = new Intent(getApplicationContext(), targetActivity);
    if (getIntent() != null && getIntent().getExtras() != null) {
        intent.putExtras(getIntent().getExtras());
    }
    return intent;
}
```

(Replace `new Intent(getApplicationContext(), targetActivity)` with `createForwardIntent(targetActivity)`).

3. Verification & Testing Checklist (Logcat)

- **FCM_TOPIC:** Check logcat for `Subscribed to 'all_users' topic successfully`
- **FCM_BACKEND:** Check logcat for `FCM token registered with backend: {"status":true,...}`
- **Notification Banner:** Trigger test slot or campaign from admin panel and verify expandable BigPicture banner displays.